Peer Architecture for sky-db

1. Overview

sky-db is a Datomic-like database library. Each peer maintains an append-only transaction log and derives a queryable index from the transactions it has seen. Peers exchange transactions over iroh-docs, and each peer's index converges to the same state as it learns of new transactions.

The design rests on four pillars:

1. **iroh-docs** as the sync protocol (a "meta protocol" over `iroh-blobs` + `iroh-gossip`).
2. A **hash chain** (`prev` pointer per transaction) for causality, with a deterministic **linearization** (topological sort + hash tiebreak) for total order.
3. **Query-time merge** (last-writer-wins by linearized order) instead of write-time rejection.
4. **Schema-as-data**: the schema is itself a set of transactions, initialized by the creating peer and appendable/updatable by any peer.

The central invariant: **the trie is append-only and never reorders; only the linearized order changes.** This is what makes "rebuild the index when I learn of a peer's transaction" cheap.

2. The log: iroh-docs

`iroh-docs` is the right layer. It is a "meta protocol" that composes the two things the log needs:

- **`iroh-blobs`** — content-addressed storage (BLAKE3). Each transaction is a blob; peers transfer only the bytes the other side is missing.
- **`iroh-gossip`** — epidemic-broadcast-tree pub/sub. This is the *"rebuild the index when I become aware of new transactions"* trigger: `doc.subscribe()` yields a live stream of new-entry events.
- **`iroh-docs`** itself — a key-value replica whose sync is *range-based set reconciliation* (the Aljoscha Meyer paper). That is the "converge on the same set of entries" primitive, and it handles offline peers / catch-up on startup, not just live gossip.

Mapping: **one replica per database, one entry per transaction.** Because entries are append-only and never overwritten, the last-writer-wins CRDT semantics never fire — each `(author, key)` is written once.

Why not the alternatives

Option	Verdict
Automerge	Not an iroh protocol; a JSON-like CRDT for collaborative editing with rich merge semantics. Wrong shape for an append-only log.
Streaming (raw `iroh-net` streams)	The low-level building block. You'd hand-roll set reconciliation + notification + persistence, reimplementing what docs already does.
`iroh-gossip` alone	Ephemeral; no history, no delivery guarantee to offline peers.
`iroh-blobs` alone	Content-addressed transfer but no "what's new" notification and no set reconciliation.

3. Core model

The log entry (transaction)

```
struct Tx {
    prev: Option<Hash>,   // hash of the tip this builds on
    author: AuthorId,     // iroh signer identity
    nonce: [u8; 16],      // random, per transaction
    datoms: Vec<Datom>,   // resolved (concrete eins), including tx metadata datoms
}
// iroh-docs entry key   = BLAKE3(serialize(Tx))   (content-addressed)
// iroh-docs entry value = serialized Tx bytes      (stored as a blob)
```

There is no global transaction counter. The `prev` pointer is the only ordering information a peer asserts.

Linearization (total order)

Every peer derives the same total order from the set of entries:

```
a < b  iff  a is an ancestor of b                (causality: follow prev pointers)
        OR  a,b concurrent and hash(a) < hash(b)  (deterministic fork tiebreak)
```

This is a topological sort with a hash tiebreak. It is a valid total order and a pure function of the log — no negotiation, no rejection. The index of each `Tx` in this order is its **linearized position**.

Merge (query-time, not write-time)

Because every datom is kept, "current value" is resolved when you read, by picking the max-position tx per `(e, a, v)`:

- **Cardinality `One`** — the single value whose latest tx has `dir == In` (last-writer-wins).
- **Cardinality `Many`** — all values whose latest tx has `dir == In` (set union, with add/del resolved by linearized order).

"No rejection" vs. "no conflict"

A hash chain never rejects at the log level — forks are first-class, and every peer's append is always valid. But "no rejection" does not mean "no conflict": concurrent writes to the same datom are **detectable and deterministically resolvable**, not rejected. The cost is that a "successful" write can be semantically overridden by a concurrent one, deterministically. That is the CRDT tradeoff, and it is the right one for a peer architecture.

4. Ordering & clocks

The distinction that matters is **causality** (what happened-before what) vs. **total order** (the linear sequence you replay).

Mechanism	What it captures	Total order?	Detects concurrency?
Hash chain (`prev` pointer)	full causal DAG (happens-before)	no (partial order)	yes — forks *are* concurrency

Vector clock	same causal partial order, compactly	no	yes
Lamport clock	logical counter, `max(seen)+1`	yes (with author tiebreak)	no
Wall-clock timestamp	nothing causal	yes, but can *violate* causality	no

- A **hash chain** and a **vector clock** are two representations of the same happens-before relation; the DAG is the explicit version, the vector clock the compact summary.
- A **Lamport clock** is a *logical* counter, not a wall-clock timestamp. Wall clocks skew, so they can stamp causally-related transactions in the wrong order.
- **Concurrency is about ancestry, not distance.** Two transactions are concurrent iff neither is an ancestor of the other. "Same distance from the MRCA" is a proxy, not the criterion — two transactions at *different* distances can still be concurrent.

Content hash vs. linearized position

Each transaction has two identities:

- **Content hash** — stable, globally unique, never changes.
- **Linearized position** — derived, and it *shifts* when a new concurrent transaction with a smaller hash is discovered late.

Key the index on the **content hash** (stable), not the position. If you keyed on position, a late fork would reorder everything and force an index rebuild, breaking the append-only property. The linearized order is a *separate, cheaply-updatable* structure.

5. Entity identity

Nonce-derived eins

Temp entities are resolved to concrete eins at transaction-creation time. The scheme is nonce-derived:

```
Ein::user(nonce) = hash(nonce) & 0x3FFF_FFFF   // 30-bit (trie key width)
```

Deriving from `prev` alone is **not** enough: two peers forking from the same tip would derive the same ein for the same temp name. The ingredients that guarantee uniqueness:

Ingredient	Disambiguates	Caveat
`prev` (tip hash)	different tips	fails on forks from the same tip
`author`	different peers forking	fails if one author writes concurrently (two devices)
per-author `seq`	every tx from one author	reintroduces a counter (per-author, no coordination)
random `nonce` (128-bit)	everything	needs a random source; nonce stored in the tx

The 30-bit birthday bound

`Ein` is `i32` (non-negative), and trie keys are 30-bit (`TrieKey::MASK = 0x3FFF_FFFF`). Deriving an ein from a hash means truncating to 30 bits, which reintroduces collision risk via the birthday bound: ~50% chance of a collision around 2^15.5 ≈ **46,000 entities**. Fine for small databases, but a real ceiling. Options: accept the bound (detect-and-re-derive on collision), or widen `Ein` to `u64` (ripples through the trie key encoding).

Note the asymmetry with `val_table` : values are content-addressed (`Vid` derived from `universal_hash::hash(&bytes, 0)` with linear probing), which works because `val_table` is single-writer at insert time. For eins in a peer model you *cannot* probe — two peers can't coordinate on "is this id taken" — so the derivation must be collision-free **by construction**.

6. Transaction entities

Every transaction is itself an entity (the Datomic model). Its metadata is ordinary datoms, queryable with the same `find` / `pull` machinery.

What it buys

1. **Unifies tx identity with entity identity.** The "tx" key in the deeper index is just the transaction entity's `Ein` — no separate hash-consing table.
2. **Metadata becomes queryable.** `author` , `prev` , `nonce` , `instant` are attributes on the transaction entity.
3. **The log entry and the entity line up.** The iroh-docs entry key (content hash) identifies the transaction; the transaction entity is its in-db representation.

The shape

```
// per transaction, at creation time:
nonce   = random 128-bit
tx_ein  = hash(nonce)                       // nonce-derived, like all other eins
datoms  = user datoms
        + [:db/add tx_ein :tx/author  author]
        + [:db/add tx_ein :tx/prev    prev_hash]
        + [:db/add tx_ein :tx/nonce   nonce]
        + [:db/add tx_ein :tx/instant now]   // optional
content_hash = BLAKE3(serialize(prev, author, nonce, datoms))   // log entry key
```

Two subtleties

1. **The content hash must not be a datom.** Storing `[:db/add tx_ein :tx/hash content_hash]` would make the hash depend on a datom that contains the hash — a self-reference. The content hash stays *external* identity; the entity's `Ein` is nonce-derived, not hash-derived. That is why nonce-derived eins matter: they break the circularity.
2. **The linearized position should not be a datom.** It shifts during convergence (a late fork reorders things), and datoms should be append-only and stable. Keep the order as a *derived* structure, not as datoms.

7. Storage layout

A datom is a full 5-tuple `[e, a, v, tx, op]` , stored in two indexes:

```
EAVT: [KEY_EAVT, eid, aid, vid, tx_ein] -> U32(dir)
AEVT: [KEY_AEVT, aid, eid, vid, tx_ein] -> U32(dir)
```

Every datom is kept — there is no overwrite on cardinality-one. This buys three things:

1. `tx` **is a full-width key component** (no packed counter).
2. **History is preserved.** The log keeps every transaction; the index does too.
3. **Conflict resolution moves from write-time to query-time.** Store every datom; resolve "current value" at query time by tx order.

`with_update`

```
pub(crate) async fn with_update(
    trie: &mut SkyTrieMut,
    aid: Ein,          // already-resolved attribute ein
    ein: Ein,
    val: Val,
    dir: Dir,
    tx_ein: Ein,
) -> Result<(), TransactError> {
    let eid = ein.to_i32();
    let vid = val_table::insert(trie, val).await?;
    let eavt_key = [KEY_EAVT, eid, aid.to_i32(), vid.to_id(), tx_ein.to_i32()];
    let aevt_key = [KEY_AEVT, aid.to_i32(), eid, vid.to_id(), tx_ein.to_i32()];
    let dir_val = TrieValue::U32(dir as u32);
    trie.insert_deep(eavt_key, dir_val.clone(), false).await;
    trie.insert_deep(aevt_key, dir_val, false).await;
    Ok(())
}
```

8. Schema

Schema-as-data

The schema is initialized by the peer that creates the repo, but peers can append and update it. The creating peer writes a **genesis transaction** (the root, `prev = None`) declaring the initial attributes, and every peer derives the schema by replaying the log. `Schema` / `AttrTable` are a *materialized view* of the `:db/ident` + `:db/cardinality` datoms.

Genesis transaction

Built-in attributes are *hardcoded* (fixed eins, always present), so the genesis tx only declares user attributes:

```
fn genesis(specs: &[AttrSpec], author: AuthorId) -> Tx {
    let nonce = rand_nonce();
    let tx_ein = Ein::tx(&nonce);
    let mut datoms = Vec::new();

    // declare user attributes (content-addressed eins)
    for spec in specs {
        let ein = Ein::attr(&spec.attr);                // hash(ident)
```

```
        datoms.push(datom::add(ein, db::ident(), spec.attr.clone()));
        datoms.push(datom::add(ein, db::cardinality(), spec.cardinality.into()));
    }

    // tx metadata (no :tx/prev — it's the root)
    datoms.push(datom::add(tx_ein, tx::author(), author));
    datoms.push(datom::add(tx_ein, tx::nonce(), nonce));
    datoms.push(datom::add(tx_ein, tx::instant(), now()));

    Tx { prev: None, author, nonce, datoms }
}
```

The chicken-and-egg is resolved by content-addressing: `Ein::attr("foo") = hash("foo")` is computable with no schema. The built-in attrs (`:db/ident` , `:db/cardinality` , `:tx/*`) are *not* declared in the genesis tx — their eins are constants the system already knows.

Schema struct

```
struct Schema {
    by_ident: HashMap<Attr, Ein>,        // ident -> ein   (for resolve)
    by_ein: HashMap<Ein, Cardinality>,   // ein -> cardinality (for query-time LWW)
}

impl Schema {
    fn empty() -> Self {
        let mut s = Self { by_ident: HashMap::new(), by_ein: HashMap::new() };
        // built-ins: fixed eins, always present
        s.builtin(db::ident(),      Cardinality::One);
        s.builtin(db::cardinality(), Cardinality::One);
        s.builtin(tx::author(),     Cardinality::One);
        s.builtin(tx::prev(),       Cardinality::One);
        s.builtin(tx::nonce(),      Cardinality::One);
        s.builtin(tx::instant(),    Cardinality::One);
        s
    }

    fn builtin(&mut self, ident: Attr, card: Cardinality) {
        let ein = Ein::attr(&ident);          // hash(ident), a constant
        self.by_ident.insert(ident, ein);
        self.by_ein.insert(ein, card);
    }

    fn resolve(&self, ident: &Attr) -> Option<Ein> {
        self.by_ident.get(ident).copied()
    }

    fn apply(&mut self, datoms: &[Datom]) {
        for d in datoms {
            if d.dir != Dir::In { continue; }
            let ein = d.ent.to_eid();
            if d.attr == db::ident() {
```

```
                if let Dat::Val(Val::String(ident)) = &d.dat {
                    self.by_ident.insert(Attr::from(ident.clone()), ein);
                }
            } else if d.attr == db::cardinality() {
                if let Dat::Val(v) = &d.dat {
                    self.by_ein.insert(ein, Cardinality::from(v.clone()));
                }
            }
        }
    }
}
```

`apply` recognizes schema declarations by *ident* (`"db/ident"` , `"db/cardinality"`), which is safe because those are built-in and always resolvable. It only folds `Dir::In` datoms — a `:db/ident` retraction would be a schema *deletion*, which is deferred (see §12).

Malformed vs. conflicting

Schema-as-data reintroduces a *kind* of rejection, distinct from the LWW merge:

- **Conflicting** transactions (two peers write the same datom concurrently) → resolved by LWW, never rejected.
- **Malformed** transactions (reference an ident not declared earlier in the linearized order) → *deterministically invalid*, skipped by every peer.

The second case is the "schema race": peer A references `:foo` while peer B concurrently declares `:foo` . In the linearized order, one comes first. This is **deterministic** — every peer computes the same order and reaches the same verdict — so it is not a coordination failure.

9. Values

`Val` is the value type stored in the index:

```
pub enum Val {
    U32(u32),
    U64(u64),
    String(String),
    Bytes(Vec<u8>),
}
```

Entity references are stored as `U32` (the ein's numeric value). Making `Ein` a first-class value variant is a deferred improvement (see §12).

Byte encoding

Values are hash-consed through a type-byte scheme (`val_table.rs`):

```
const VAL_TYPE_U32: u8 = 16;
const VAL_TYPE_STRING: u8 = 17;
const VAL_TYPE_U64: u8 = 18;
const VAL_TYPE_BYTES: u8 = 19;
```

```
fn bytes_from_val(val: &Val) -> Vec<u8> {
    match val {
        Val::U32(u)    => { let mut b = vec![VAL_TYPE_U32];
b.extend_from_slice(&u.to_be_bytes()); b }
        Val::U64(u)    => { let mut b = vec![VAL_TYPE_U64];
b.extend_from_slice(&u.to_be_bytes()); b }
        Val::String(s) => { let mut b = vec![VAL_TYPE_STRING];
b.extend_from_slice(s.as_bytes()); b }
        Val::Bytes(bs) => { let mut b = vec![VAL_TYPE_BYTES];
b.extend_from_slice(bs); b }
    }
}
```

The key invariant: **the encoding must be injective** — distinct `Val`s must produce distinct byte sequences, or the hash-consing table would collapse two values into one `Vid`. The type byte guarantees this. The `Vid` derivation (`universal_hash::hash(&bytes, 0) & TrieKey::MASK`) is agnostic to the payload.

10. Replay & the index

The central invariant: **the trie is append-only and never reorders; only the linearized order changes.** Replay is two separable halves — insert datoms (order-independent) and compute the order (the only part that shifts on a late fork).

```
struct Index {
    schema: Schema,                // materialized view of :db/ident +
:db/cardinality
    trie: SkyTrie,                  // append-only datom storage
    order: HashMap<Ein, u32>,       // tx_ein -> linearized position (the only
mutable part)
    tip: Option<Hash>,              // last element of the linearized order
}
```

Linearize

Each `Tx` has one `prev`, so the log is a forest (each node one parent, many children). Topological sort with a min-heap keyed by hash gives a deterministic total order:

```
fn linearize(txs: &HashMap<Hash, Tx>) -> Vec<Hash> {
    // children[prev] = hashes of txs that point at `prev`
    let mut children: HashMap<Option<Hash>, Vec<Hash>> = HashMap::new();
    for (hash, tx) in txs {
        children.entry(tx.prev).or_default().push(*hash);
    }

    // min-heap of ready txs, keyed by hash (deterministic tiebreak for forks)
    let mut ready: BinaryHeap<Reverse<Hash>> = BinaryHeap::new();
    for root in children.get(&None).into_iter().flatten() {
        ready.push(Reverse(*root));
    }
```

```
    let mut order = Vec::with_capacity(txs.len());
    while let Some(Reverse(hash)) = ready.pop() {
        order.push(hash);
        if let Some(kids) = children.get(&Some(hash)) {
            for kid in kids { ready.push(Reverse(*kid)); }
        }
    }
    debug_assert_eq!(order.len(), txs.len());   // unreachable: cycles are
impossible
    order
}
```

Two properties:

- **Cycles are impossible by construction.** A tx's hash covers its `prev`, so a cycle would require a hash to contain itself. Content-addressing gives a DAG for free.
- **The tiebreak is total and stable.** "Smallest hash" is a total order over BLAKE3 hashes, and a tx's hash never changes, so every peer computes the identical order.

The merge lives at query time

```
fn current_fills(&self, ein: Ein, attr_ein: AttrEin) -> Vec<Vid> {
    let mut latest: HashMap<Vid, (u32, Dir)> = HashMap::new();
    for (vid, tx_ein, dir) in self.all_entries(ein, attr_ein) {
        let pos = self.order[&tx_ein];
        match latest.entry(vid) {
            Entry::Vacant(e) => { e.insert((pos, dir)); }
            Entry::Occupied(mut e) => {
                if pos > e.get().0 { *e.get_mut() = (pos, dir); }
            }
        }
    }
    latest.into_iter()
        .filter(|(_, (_, dir))| *dir == Dir::In)
        .map(|(vid, _)| vid)
        .collect()
}
```

Cardinality-one is "the single vid whose latest tx has `dir == In`"; cardinality-many is "all such vids." This is the LWW-by-linearized-order merge, a pure function of `(trie, order)`.

Running-schema replay

Because the schema is data, replay builds it incrementally, in linearized order. The ordering inside the loop is the crux — a transaction resolves against the schema *before* its own declarations are folded in, so "declare before use" holds across transactions, not within one:

```
async fn replay(txs: Vec<Tx>) -> anyhow::Result<Index> {
    let map: HashMap<Hash, Tx> = txs.into_iter().map(|tx| (tx.hash(),
tx)).collect();
    let order = linearize(&map);
```

```
    let mut schema = Schema::empty();
    let mut trie = SkyTrie::new();
    let mut order_map = HashMap::new();

    for (i, hash) in order.iter().enumerate() {
        let tx = &map[hash];
        let tx_ein = Ein::tx(&tx.nonce);
        order_map.insert(tx_ein, i as u32);

        // 1. resolve + insert every datom against the *current* schema
        for datom in &tx.datoms {
            let Some(aid) = schema.resolve(&datom.attr) else {
                continue;                // malformed: undeclared ident, skip
            };
            with_update(&mut trie, aid, datom.ent.to_eid(), datom.dat.clone(),
                        datom.dir, tx_ein).await?;
        }

        // 2. fold this tx's schema declarations into the running schema
        schema.apply(&tx.datoms);
    }

    Ok(Index { schema, trie, order: order_map, tip: order.last().copied() })
}
```

The `continue` on unresolved idents is the "deterministically skip malformed transactions" policy, applied at the *datom* level. To skip the whole transaction atomically, resolve all idents first and bail on the first miss — a policy choice, not a structural one.

Incremental update

The full `replay` above is the startup path. For a live arrival:

- **New tx builds on the current tip** (common case): append its datoms, append it to the end of `order` — O(1) to the order.
- **New tx is a fork** (its `prev` is an earlier tx): insert its datoms, then re-linearize — the order shifts, but the trie does *not*. Only `self.order` is recomputed (O(n log n), rare).

So "rebuild the index" is really "append datoms + recompute the order map." The expensive part (the trie) is never rewritten.

Caveat: a fork must re-resolve against the historical schema. A fork's linearized position is in the *middle* of the order, so its datoms must be resolved against the schema *as of that position*, not the current (tip) schema. If a late fork references an ident declared after its position, a naive incremental apply (resolving against the tip schema) would insert a datom that a fresh full replay would have skipped as malformed — a divergence. Correct incremental handling therefore requires either re-resolving the fork's datoms against the schema at its position (a history of schema states), or falling back to a full replay when a fork arrives. The "append-only trie" claim holds for the *common* case (a tx building on the tip); the fork case is where the incremental path gets subtle.

11. Live updates & snapshots

The live object and the query object are separate.

`Peer` — the live object

```
struct Peer {
    log: Log,                       // iroh-docs wrapper (endpoint + blobs + gossip
+ docs + router)
    index: Arc<RwLock<Index>>,      // the shared, mutable index
    task: JoinHandle<()>,           // the background replay loop
}
```

`Peer` owns the log and the index. It exposes:

- **`transact(datoms)`** — mints a nonce, resolves temp entities, builds a `Tx` (with the peer's `AuthorId` and the current tip as `prev`), appends it to the log, and applies it to the index.
- **`ticket()`** — recreates a fresh `DocTicket` on demand (the ticket embeds the current endpoint address, so it is never persisted).
- **`snapshot()`** — hands out a stable, immutable `Pod`.

A background task drains `log.subscribe()` and, on each `ContentReady` / `PendingContentReady` / `SyncFinished` event, re-reads the log and full-replays it into the shared index. This is the "rebuild the index when I learn of a peer's transaction" path.

`Pod` — the stable snapshot

```
struct Pod {
    schema: Schema,
    trie: SkyTrie,
    order: HashMap<Ein, u32>,
}
```

`Pod` is an immutable snapshot of the index (schema + trie + order), handed out on demand. All queries (`find`, `pull`, `ev_iter`, `list_attributes`, …) run against a `Pod`. Because the trie is `Arc`-backed, a snapshot is cheap, and a held `Pod` stays valid even as the `Peer` continues to advance.

Entry points

```
create(path, db_spec) -> Peer   // new repo: keypair + genesis tx + namespace
load(path)            -> Peer   // existing repo: read keypair + namespace, open,
replay
join(path, ticket)    -> Peer   // join a peer's repo: import ticket, sync, replay
```

The path holds two files — `keypair` (the peer identity) and `namespace` (the repo id). The `DocTicket` is never persisted; it is recreated on demand via `Peer::ticket()`.

12. Deferred / open questions

1. **System/user distinction.** Reserve a bit of the `Ein` (e.g. bit 29) as a system flag, or use a marker attribute. Trivially addable later; it is a filter predicate in `list_entities` / `find`. Tradeoff: a bit flag drops user space to 29 bits (~23k birthday bound); a marker attribute costs a query per entity.

2. **`Ein` as a first-class value.** Entity references are currently stored as `U32`; a dedicated `Val::Ein` variant would disambiguate them from integers. The `find`/query DSL changes are deferred.
3. **`find`/query-layer changes** for the `Ein` / `U32` split.
4. **Schema deletion policy.** `apply` only folds `Dir::In`, so the schema is effectively append-only. Attribute deletion is genuinely hard in a replicated log (it orphans existing datoms); "deprecate, don't delete" is the likely answer.
5. **`Ein` width.** 30-bit birthday bound (~46k entities). Widen to `u64` if needed; ripples through the trie key encoding.
6. **`tip()` semantics.** The tip is `order`'s last element, used as the next `prev`. A peer's tip is "the max of what I've seen so far" — it advances as new txs arrive, which is fine because `prev` only records what the peer knew at creation time.
7. **Incremental replay.** The §10 incremental path (append-only trie, re-linearize on forks) is a deferred optimization — the current implementation uses full replay on each sync event. The fork case's schema re-resolution caveat (§10) is the main reason it is not the first cut.

Appendix: key invariants

- The trie is append-only; only the linearized order changes.
- Content hash is external identity; never a datom (self-reference).
- Linearized position is derived; never a datom (unstable during convergence).
- Ein derivation is collision-free by construction (nonce/ident), not by probing.
- `Val` encoding is injective (type byte disambiguates).
- Cycles are impossible by construction (content-addressed `prev`).
- Schema is append-only (deletion deferred); "declare before use" across transactions.